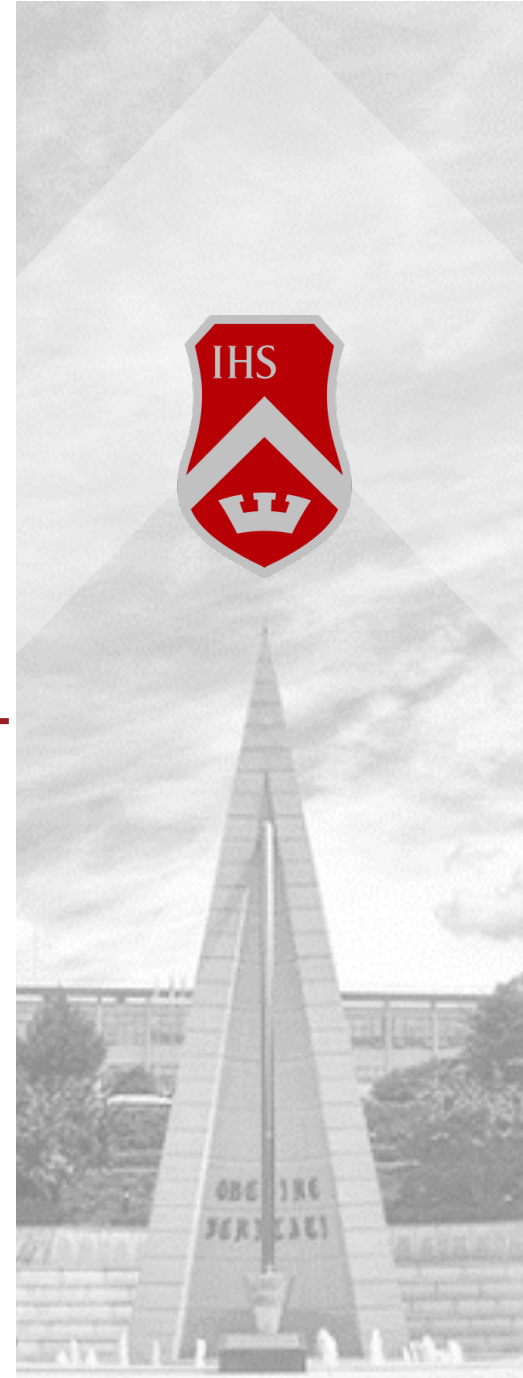


Lecture 04

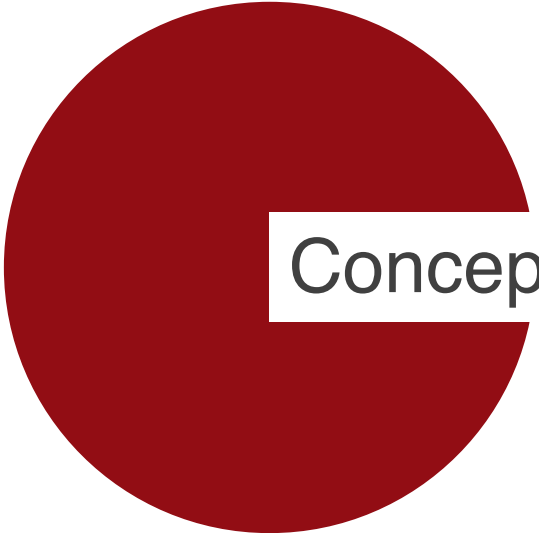
Model-based Recommender Systems (part 1)

Euhyun Moon, Ph.D.
Machine Learning System (MLSys) Lab
Computer Science and Engineering
Sogang University



Today's Class

- Model-based Recommender Systems
 - Concept of Latent Factor Models
 - Latent Factor Models for Implicit Feedbacks



Concept of Latent Factor Models

Latent Factor Models

- In memory-based recommender systems, **dimensionality reduction** methods (e.g., SVD) primarily play the role of creating a more convenient data representation for neighborhood-based methods
- In this chapter, more sophisticated methods will be discussed, and the goal of this chapter is to use dimensionality reduction techniques to **directly estimate the data (rating) matrix** in one shot

Latent Factor Models

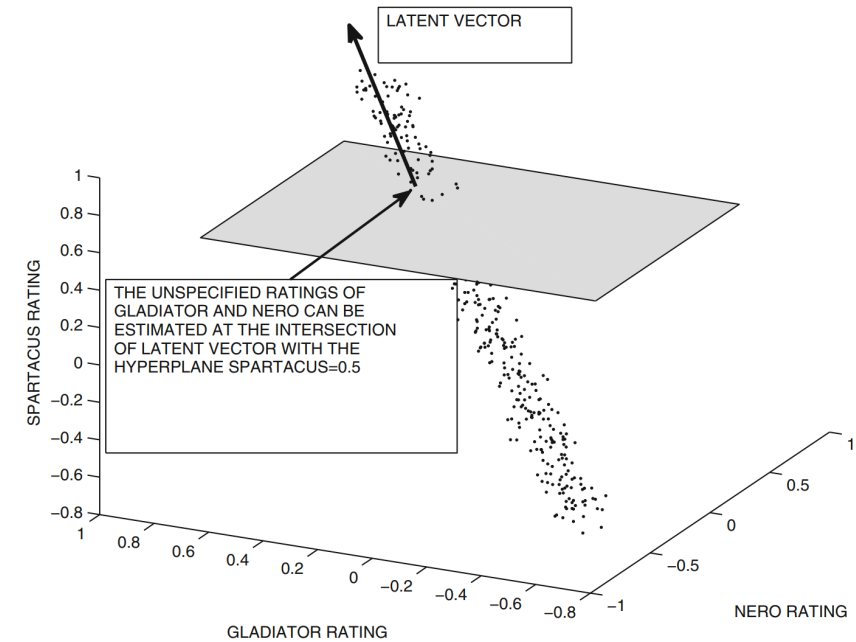
- Latent factor models are considered as state-of-the-art models in recommender systems
- Latent factor models utilize well-known dimensionality reduction methods to fill in missing entries
- The key idea behind dimensionality reduction methods is that a reduced, rotated, and fully specified representation can be robustly estimated from an incomplete data matrix

Latent Factor Models

- Dimensionality reduction methods leverage correlations between rows (users) and columns (items) to generate a fully specified and reduced representation
- The use of these correlations is fundamental to all collaborative filtering methods, whether they are neighborhood-based or model-based
- **Matrix factorization** techniques provide an efficient way to leverage all row and column correlations simultaneously to estimate the entire data matrix

Geometric Intuition for Latent Factor Models

- Data with p correlations can be approximately represented on a p -dimensional hyperplane
- For example, if users' ratings for the three movies—Spartacus, Gladiator, and Nero—are positively correlated, the data will primarily align along a one-dimensional line (i.e., a latent factor)
- The unspecified ratings for Gladiator and Nero can be estimated at the intersection of the latent vector with the hyperplane where Spartacus is set to 0.5, as specified by the user



Geometric Intuition for Latent Factor Models

- It is assumed that a specified data matrix is available for estimating the relevant latent vector
- The data matrix does not need to be fully specified to estimate the dominant latent vectors
- The advantage of the latent factor approach lies in its ability to estimate latent vectors even when data is missing
- We must use a partially specified (missing) dataset to recover the low-dimensional hyperplane that approximately represents the data

Low-Rank Intuition for Latent Factor Models

- **Factorization** is a more general method for approximating a matrix when it is subject to dimensionality reduction due to correlations between its columns or rows
- Most dimensionality reduction methods can be represented as **matrix factorizations**

Low-Rank Intuition for Latent Factor Models

- Matrix factorization involves decomposing a matrix into the product of two or more matrices, typically to simplify or analyze the original matrix
- The key idea is that any matrix R of size $n \times m$ can always be expressed as the product of rank- k factors

$$R = UV^T$$

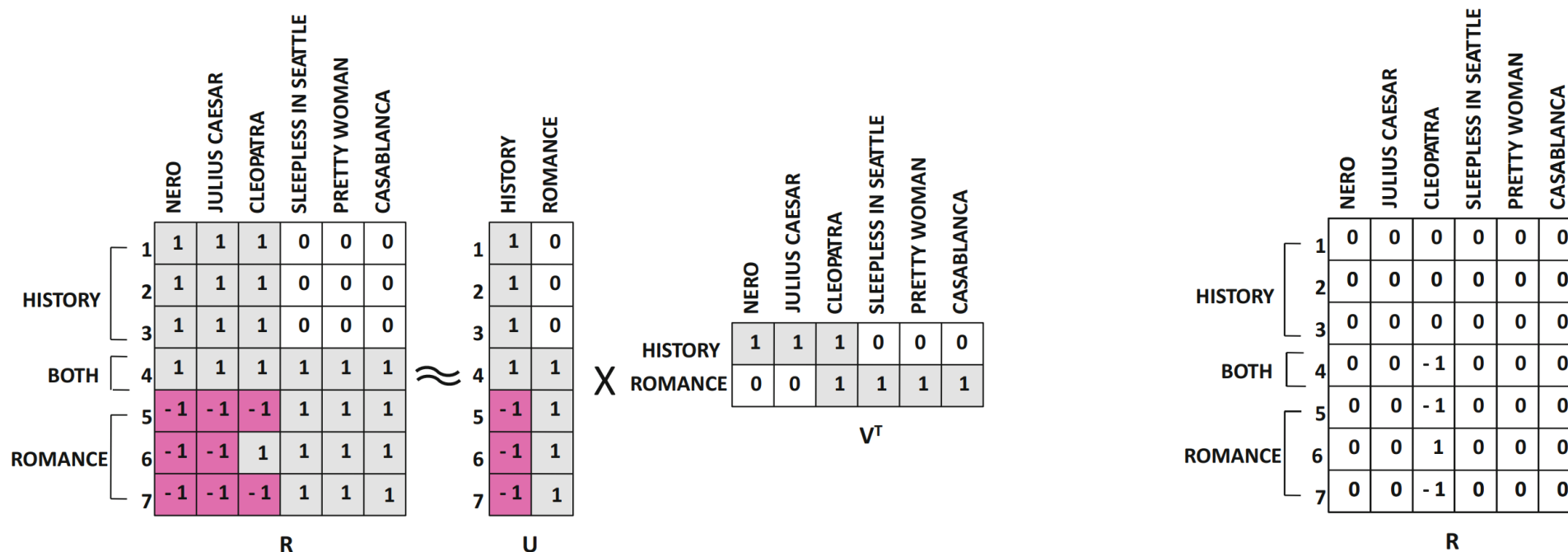
$$\text{where } R \in \mathbb{R}^{n \times m}, U \in \mathbb{R}^{n \times k}, V \in \mathbb{R}^{m \times k}$$

- When the matrix R exhibits correlations greater than k , it can often be approximately expressed as the product of rank- k factors

$$R \approx UV^T$$

Low-Rank Intuition for Latent Factor Models

- For example, if the following rating matrix of size 7×6 has two rank correlations, it can be approximately factorized into rank-2 factors



Basic Matrix Factorization Principles

- In the basic matrix factorization model, the $n \times m$ rating matrix R is approximately factorized into a matrix $U \in \mathbb{R}^{n \times k}$ and a matrix $V \in \mathbb{R}^{m \times k}$

$$R \approx UV^T$$

- Each row of U and V represents a latent factor
- The i -th row $\bar{u}_i$ of U is referred to as a user factor, while the j -th row $\bar{v}_j$ of V is referred to as an item factor
- Each rating r_{ij} in R can be approximately expressed as the dot product of the i -th user factor and the j -th item factor

$$r_{ij} \approx \bar{u}_i \cdot \bar{v}_j$$

Basic Principle of Matrix Factorization

- The latent factors $\bar{u}_i = (u_{i1}, \dots, u_{ik})$ and $\bar{v}_j = (v_{j1}, \dots, v_{jk})$ can be viewed as the affinities of users and items for k different concepts

$$r_{ij} \approx \sum_{s=1}^k u_{is} \cdot v_{js} = \sum_{s=1}^k (\text{affinity of user } i \text{ to the concept } s) \cdot (\text{affinity of item } j \text{ to the concept } s)$$

- For example, with two concepts (rank $k = 2$), these could represent the romance and historical genres

$$\begin{aligned} r_{ij} &\approx (\text{affinity of user } i \text{ to history}) \times (\text{affinity of item } j \text{ to history}) \\ &\quad + (\text{affinity of user } i \text{ to romance}) \times (\text{affinity of item } j \text{ to romance}) \end{aligned}$$

Unconstrained Matrix Factorization

- The most basic form of matrix factorization is the unconstrained case, where no constraints are imposed on the factor matrices U and V
- Strictly speaking, SVD is not an unconstrained matrix factorization because the columns of U and V must be orthogonal in SVD

→ Finding Optimal Latent Factors by Learning

Unconstrained Matrix Factorization

- Let the set of all user-item pairs (i, j) observed in matrix R be represented as the approximate product UV^T , where $U = [u_{is}]_{n \times k}$ and $V = [v_{js}]_{m \times k}$ are fully specified matrices. Then, all the entries in R can be predicted as follows:

$$\hat{r}_{ij} = \sum_{s=1}^k u_{is} \cdot v_{js}$$

- The difference between the observed and predicted value of a specified entry (i, j) is given by $e_{ij} = (r_{ij} - \hat{r}_{ij}) = (r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js})$

Unconstrained Matrix Factorization

- The objective function (error or cost function) is computed only over the observed entries in S

$$\text{Minimize } \mathcal{L} = \frac{1}{2} \sum_{(i,j) \in S} e_{ij}^2 = \frac{1}{2} \sum_{(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js} \right)^2 \text{ subject to: no constraints on } U \text{ and } V$$

- Here, u_{is} and v_{js} are the unknown (trainable) variables that need to be learned in order to minimize the objective function $\mathcal{L}$

Unconstrained Matrix Factorization

- This can be achieved using gradient descent, which requires computing the partial derivatives of $\mathcal{L}$ with respect to the variables u_{is} and v_{js}

$$\frac{\Delta \mathcal{L}}{\Delta u_{is}} = \sum_{(i,j) \in S} (r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js})(-v_{js}) = \sum_{(i,j) \in S} e_{ij} \cdot (-v_{js})$$

$$\frac{\Delta \mathcal{L}}{\Delta v_{js}} = \sum_{(i,j) \in S} (r_{ij} - \sum_{s=1}^k u_{is} \cdot v_{js})(-u_{is}) = \sum_{(i,j) \in S} e_{ij} \cdot (-u_{is})$$

Gradient Descent for Unconstrained Matrix Factorization

Algorithm GD (rating matrix R and learning rate α)

begin

Randomly initialize matrices U and V ;

$S = \{(i, j): r_{ij} \text{ is observed}\};$

while not converge **do**

begin

Compute each error e_{ij}

for each user-item pair (i, j) **do** $u_{is}^+ \leftarrow u_{is} + \alpha \cdot \sum_{(i, j) \in S} e_{ij} \cdot v_{js}$

for each user-item pair (i, j) **do** $v_{js}^+ \leftarrow v_{js} + \alpha \cdot \sum_{(i, j) \in S} e_{ij} \cdot u_{is}$

for each user-item pair (i, j) **do** $u_{is} \leftarrow u_{is}^+$

for each user-item pair (i, j) **do** $v_{js} \leftarrow v_{js}^+$

end

end

Stochastic Gradient Descent

- This aforementioned method is referred to as the batch update method
- An important observation is that the updates are linear functions of the errors in the observed entries of the rating matrix
- The update can also be performed by decomposing it into smaller components (mini-batch) associated with the errors in individual observed entries, rather than considering all entries at once
- This update can be stochastically approximated based on the error in a randomly chosen observed entry (i, j)

$$u_{is} \leftarrow u_{is} - \alpha \cdot \left[\frac{\Delta \mathcal{L}}{\Delta u_{is}} \right]_{mini-batch}$$

$$v_{js} \leftarrow v_{js} - \alpha \cdot \left[\frac{\Delta \mathcal{L}}{\Delta v_{js}} \right]_{mini-batch}$$

Stochastic Gradient Descent

Algorithm SGD (rating matrix R and learning rate α)

begin

Randomly initialize matrices U and V ;

$S = \{(i, j): r_{ij} \text{ is observed}\};$

while not converge **do**

begin

Randomly sampled batch $B \subseteq S$

Compute each error e_{ij}

for each user-item pair $(i, j) \in B$ **do** $u_{is}^+ \leftarrow u_{is} + \alpha \cdot \sum_{(i,j) \in B} e_{ij} \cdot v_{js}$

for each user-item pair $(i, j) \in B$ **do** $v_{js}^+ \leftarrow v_{js} + \alpha \cdot \sum_{(i,j) \in B} e_{ij} \cdot u_{is}$

for each user-item pair $(i, j) \in B$ **do** $u_{is} \leftarrow u_{is}^+$ and $v_{js} \leftarrow v_{js}^+$

end

end

Regularization

- A small observed set S of ratings can lead to **overfitting**
- A common approach to address this problem is **regularization**
- Regularization aims to discourage very large values of the coefficients in U and V

Regularization

- A regularization term is added to the objective function, where $\lambda > 0$ is the regularization hyperparameter

$$\text{Minimize } \mathcal{L} = \frac{1}{2} \sum_{(i,j) \in S} e_{ij}^2 + \frac{\lambda}{2} \sum_{i=1}^n \sum_{s=1}^k u_{is}^2 + \frac{\lambda}{2} \sum_{j=1}^m \sum_{s=1}^k v_{js}^2$$

$$\frac{\Delta \mathcal{L}}{\Delta u_{is}} = \sum_{(i,j) \in S} e_{ij} \cdot (-v_{js}) + \lambda u_{is}$$

$$\frac{\Delta \mathcal{L}}{\Delta v_{js}} = \sum_{(i,j) \in S} e_{ij} \cdot (-u_{is}) + \lambda v_{js}$$

Alternating Least Squares

- The stochastic gradient method is an efficient approach for optimization
- However, it is quite sensitive to both the initialization and the selection of step sizes
- Other optimization methods include ***alternating least squares (ALS)***, which is generally more stable

Alternating Least Squares

- The basic idea of ALS is to use the following iterative method, starting with an initial set of matrices U and V
 - With U fixed, update the latent factors in V
 - With V fixed, update the latent factors in U
 - Repeat the above steps until convergence

Incorporating User and Item Biases

- A variation of the unconstrained model incorporates variables that can learn user and item biases
- Each user u_i has an associated variable o_i , which indicates the general bias of the user in rating items
- Similarly, the variable p_j denotes the bias in the rating of item v_j

Incorporating User and Item Biases

- The predicted rating for entry (i, j) is given by the following expression:

$$\hat{r}_{ij} = \mathbf{o}_i + \mathbf{p}_j + \sum_s^k u_{is} \cdot v_{js}$$

- The error e_{ij} for an observed entry (i, j) is given by the following expression:

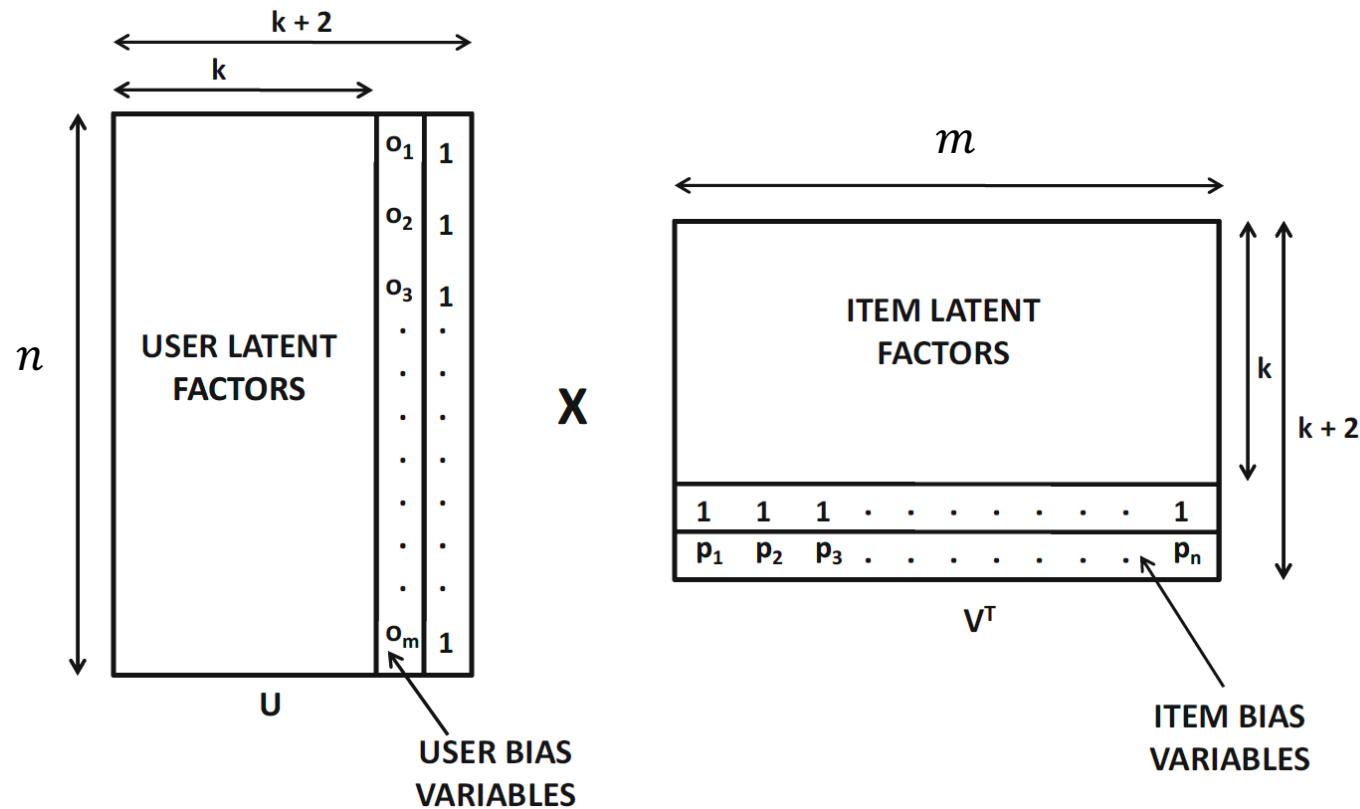
$$e_{ij} = r_{ij} - \hat{r}_{ij} = r_{ij} - \mathbf{o}_i - \mathbf{p}_j - \sum_{s=1}^k u_{is} \cdot v_{js}$$

- The objective function to be minimized is modified as follows:

$$\text{Minimize } \mathcal{L} = \frac{1}{2} \sum_{(i,j) \in S} e_{ij}^2 + \frac{\lambda}{2} \sum_{i=1}^n \sum_{s=1}^k u_{is}^2 + \frac{\lambda}{2} \sum_{j=1}^m \sum_{s=1}^k v_{js}^2 + \frac{\lambda}{2} \sum_{i=1}^n \mathbf{o}_i^2 + \frac{\lambda}{2} \sum_{j=1}^m \mathbf{p}_j^2$$

Incorporating User and Item Biases

- Rather than using separate bias variables o_i and p_j for users and items, we can increase the size of the factor matrices to incorporate these bias variables



Incorporating User and Item Biases

- In fact, it has been shown that using only the bias variables can often yield reasonably accurate rating predictions
- A significant portion of the ratings can be explained by user generosity and item popularity, rather than by specific personalized user preferences for items

Incorporating Implicit Feedback

- Typically, implicit feedback scenarios correspond to unary ratings matrices, where users express their interests through purchasing items
- Even when users explicitly rate items, the selection (identity) of items they choose to rate can be considered implicit feedback
- In other words, significant predictive value is captured by the choice of items that users rate, regardless of the actual rating values

Incorporating Implicit Feedback

- Example

$$\underbrace{\begin{pmatrix} 1 & -1 & 1 & ? & 1 & 2 \\ ? & ? & -2 & ? & -1 & ? \\ 0 & ? & ? & ? & ? & ? \\ -1 & 2 & -2 & ? & ? & ? \end{pmatrix}}_R \Rightarrow \underbrace{\begin{pmatrix} 1/\sqrt{5} & 1/\sqrt{5} & 1/\sqrt{5} & 0 & 1/\sqrt{5} & 1/\sqrt{5} \\ 0 & 0 & 1/\sqrt{2} & 0 & 1/\sqrt{2} & 0 \\ 1/\sqrt{1} & 0 & 0 & 0 & 0 & 0 \\ 1/\sqrt{3} & 1/\sqrt{3} & 1/\sqrt{3} & 0 & 0 & 0 \end{pmatrix}}_F$$

R : actual rating matrix (explicit feedback)

F : normalized implicit feedback matrix

Incorporating Implicit Feedback

- Various frameworks, such as asymmetric factor models and SVD++, have been proposed to incorporate implicit feedback. These methods also utilize an additional item factor matrix $Y \in \mathbb{R}^{m \times k}$

Asymmetric Factor Models

- To capture implicit feedback information, the method first derives an implicit feedback matrix from the explicit rating matrix

$$\underbrace{\begin{pmatrix} 1 & -1 & 1 & ? & 1 & 2 \\ ? & ? & -2 & ? & -1 & ? \\ 0 & ? & ? & ? & ? & ? \\ -1 & 2 & -2 & ? & ? & ? \end{pmatrix}}_R \Rightarrow \underbrace{\begin{pmatrix} 1/\sqrt{5} & 1/\sqrt{5} & 1/\sqrt{5} & 0 & 1/\sqrt{5} & 1/\sqrt{5} \\ 0 & 0 & 1/\sqrt{2} & 0 & 1/\sqrt{2} & 0 \\ 1/\sqrt{1} & 0 & 0 & 0 & 0 & 0 \\ 1/\sqrt{3} & 1/\sqrt{3} & 1/\sqrt{3} & 0 & 0 & 0 \end{pmatrix}}_F$$

$$R \approx [FY]V^T$$

- This simple approach often provides excellent results, contains fewer parameters than the user-factor matrix U , and allows for the incorporation of other types of independent implicit feedback

Asymmetric Factor Models

- An additional advantage of this model is that it requires no user parameterization, allowing it to perform well with out-of-sample (cold-start) users
- The item-based parameterization of asymmetric factor models also provides the merit of explainability
 - $[FY]V^T$ can be rewritten as $F[YV^T]$
 - The matrix YV^T of size $m \times m$ can be viewed as an item-to-item prediction matrix, indicating which items previously consumed by the user contribute most to the prediction

SVD++

- In SVD++, the implicit user-factor matrix FY is used only to adjust the explicit user-factor matrix U rather than to generate it
- The reconstructed rating matrix R of size $n \times m$ is as follows:

$$R \approx (U + FY)V^T$$

- Although this model is referred to as SVD++, the name is somewhat misleading because the basis vectors of the factorized matrices are not orthogonal
- The term “SVD” is often applied loosely in the literature on latent factor models

Non-negative Matrix Factorization (NMF)

- **Non-negative matrix factorization (NMF)** can be used for rating matrices that contain only non-negative values
- The primary difference from other forms of matrix factorization is that the factors U and V must be non-negative
- The optimization formulation for non-negative matrix factorization is defined as follows:

$$\text{Minimize } \mathcal{L} = \frac{1}{2} \|R - UV^T\|^2$$

subject to: $U \geq 0, V \geq 0$

Non-negative Matrix Factorization (NMF)

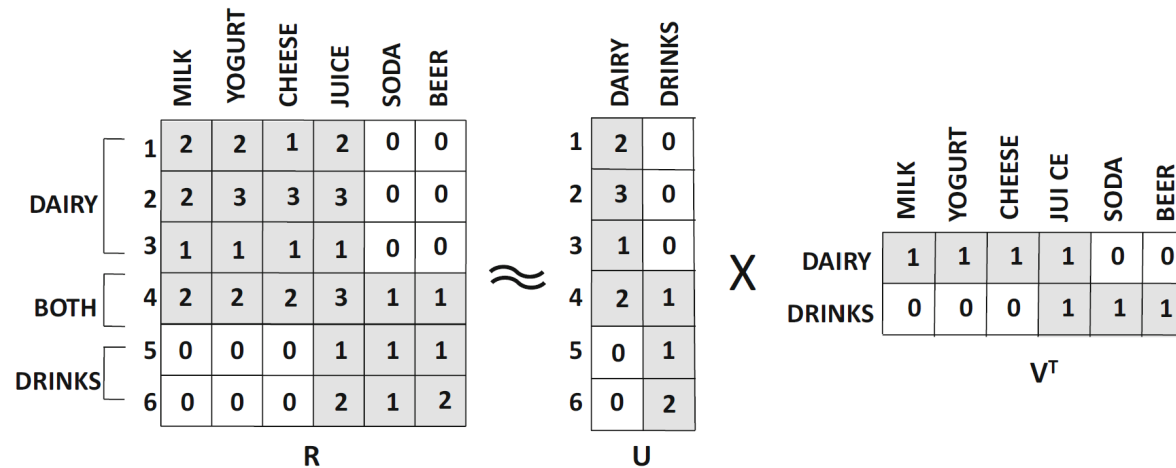
- To ensure positive values for the factors U and V , the parameter optimization proceeds as follows:

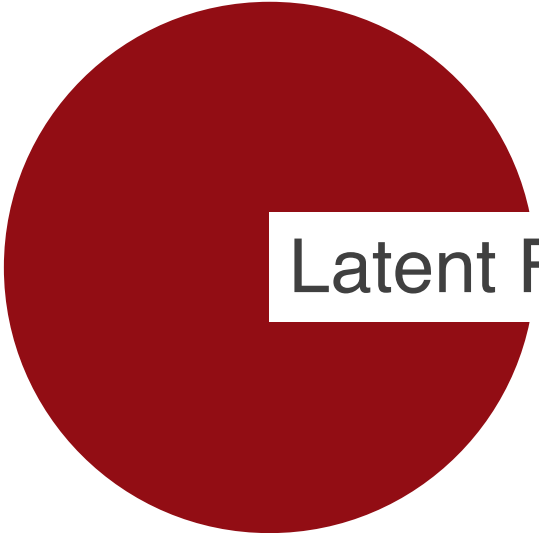
$$u_{ij} \leftarrow \max \left\{ \left[\frac{(RV)_{ij} - \lambda_1 u_{ij}}{(UV^T V)_{ij} + \epsilon} \right] u_{ij}, 0 \right\}$$

$$v_{ij} \leftarrow \max \left\{ \left[\frac{(R^T U)_{ij} - \lambda_2 v_{ij}}{(V U^T U)_{ij} + \epsilon} \right] v_{ij}, 0 \right\}$$

Non-negative Matrix Factorization (NMF)

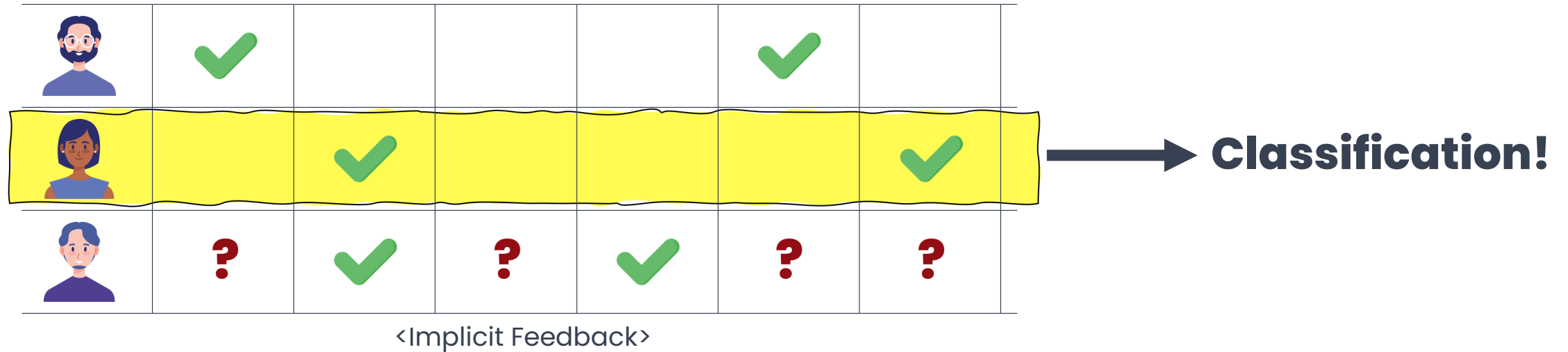
- The primary advantage of this approach is the high level of interpretability it offers in understanding user-item interactions
- This is one of the reasons NMF is frequently used in clustering
- Each of these clusters can often be examined to semantically interpret the associations between users and items





Latent Factor Models for Implicit Feedbacks

Recap: Implicit Feedback



Naïve Approach: Considering Implicit Feedback as Explicit Feedback

- In the real world, many recommendation systems are designed to address implicit feedback
- These systems also use latent factor models to capture correlations between users and items
- The naïve approach involves using traditional matrix factorization methods by treating implicit feedback as explicit feedback (e.g., assigning ratings of 0 or 1)

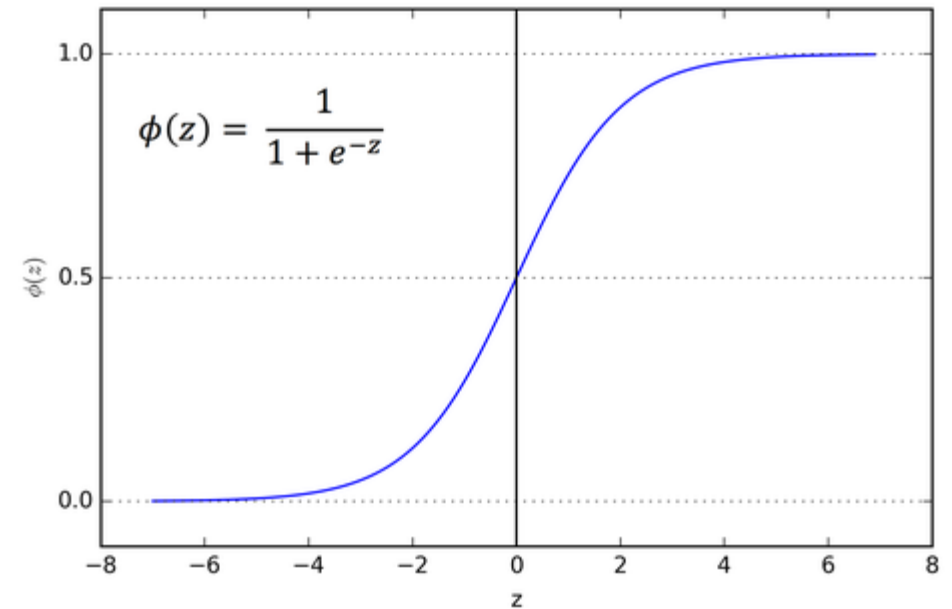
Naïve Approach: Considering Implicit Feedback as Explicit Feedback

- This naïve approach has certain drawbacks:
 - The predicted ratings may fall outside the range of $[0, 1]$
 - A linear combination of variables cannot capture the nonlinear relationships between users and items

Logistic Regression

- **Logistic regression** is a statistical method used for binary classification, applied when the dependent variable is categorical with only two possible outcomes, typically labeled as 0 and 1
- Logistic regression utilizes the logistic function, also known as the sigmoid function:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Logistic Regression

- A simple form of rating prediction using logistic regression is defined as follows:

$$x = o_i + p_j + \sum_s^k u_{is} \cdot v_{js}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$p(y = 1 | (u_i, v_j)) = \hat{r}_{ij} = \sigma(x) = \sigma\left(o_i + p_j + \sum_s^k u_{is} \cdot v_{js}\right)$$

Logistic Regression

- Advantages of logistic regression with the logistic function include:
 - Complex function approximation
 - Real-world data and relationships are often nonlinear
 - Nonlinear activation functions are essential for capturing and modeling nonlinear dependencies and interactions within the input data
 - Representation Power
 - This enhanced representational power is essential for capturing intricate patterns and features within the data

Logistic Regression: Optimization

- The logistic regression model is optimized by minimizing the following log loss function, rather than using mean squared error (MSE):

$$\mathcal{L} = -\frac{1}{|D|} \sum_{d_i \in D} y_i \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

where $d = (u, v, y)$

- In binary classification, the log loss function is equivalent to the cross-entropy loss function